

BDX Slow Control User and Developer Manual

BDX Slow Control

Document status: Development documentation for the `devel` branch

Repository: [vallarinosimone1992/bdx-slow-control](https://github.com/vallarinosimone1992/bdx-slow-control)

Revision date: July 23, 2026

Contents

I	User manual	1
1	Purpose and audience	1
2	System summary	1
3	Before starting	1
4	Starting the system	2
4.1	Raspberry environment IOC	2
4.2	Complete operator stack	2
5	Using Phoebus	3
6	Operating the low-voltage supplies	3
7	Operating the chiller	4
8	Environmental temperatures	4
9	Viewing archived history	5
10	Operator command implementation reference	5
11	Stopping the software	6
12	Troubleshooting reference	6
II	Developer guide	7
13	Scope	7
14	Design goals	7
15	Architecture	8
15.1	Layering	8
15.2	Runtime model	8
15.3	Current deployed split	8
16	Repository layout	9
17	Development environment	9
17.1	Requirements	9
17.2	Useful checks	9
18	Configuration model	10
18.1	Profiles	10
18.2	Common server settings	10
18.3	Device entries	10
18.4	Runtime environment	11

19 PV contract	11
19.1 Naming convention	11
19.2 Managed IOC health PVs	11
20 Driver layer	12
20.1 Interfaces and state objects	12
20.2 Simulation first	12
20.3 Adding a hardware backend	13
21 IOC layer	13
21.1 ManagedIOC	13
21.2 Staged operator requests	13
21.3 Commands	14
22 Builders and CLI entry points	14
22.1 Adding a new subsystem	14
23 Phoebus generation	15
23.1 Generated resources	15
23.2 Common generation commands	15
23.3 Plot and archive settings	15
24 Archiver Appliance integration	16
25 Operator command layer	17
26 Raspberry environment IOC	18
27 Run temperature export	18
28 Testing strategy	19
28.1 Required test categories	19
28.2 Pre-commit checklist	19
29 Logging and diagnostics	19
30 Git workflow and stable releases	20
31 Building these manuals	21
32 Change checklist	21
A Principal environment variables	21
B Principal console scripts	22

Part I

User manual

1 Purpose and audience

This part is the detailed user manual for operators who need to start, monitor, and stop the BDX slow-control prototype without modifying its software. It explains the normal Ubuntu workflow, the Phoebus operator displays, the low-voltage power supplies, the chiller, the Raspberry Pi temperature IOC, archived history, and common faults.

The main Ubuntu host runs the global, low-voltage power-supply, and chiller IOC groups. A Raspberry Pi runs the environmental temperature IOC. Phoebus is the graphical interface, while EPICS Archiver Appliance stores selected process variables for later retrieval.

Operational safety

The slow-control software is not a substitute for hardwired protection, approved operating procedures, or direct verification at the equipment. Before enabling an output or changing a setpoint, confirm the intended device, channel, limits, load state, and detector condition.

2 System summary

Component	Address	Function
Main IOC host	172.22.50.2	Global state, LV1, LV2, and chiller IOC groups
Raspberry Pi	172.22.50.10	MCP9808 environmental temperature IOC
LV1	172.22.50.20:9221	TTi CPX400DP low-voltage supply
LV2	172.22.50.21:9221	TTi CPX400DP low-voltage supply
Chiller	172.22.50.60:54321	LAUDA ECO Silver RE 1225 S
Archiver	127.0.0.1:17665-17668	Local Archiver Appliance services

The default main-host profile is stored in `config/profiles/default`. The Raspberry profile is stored separately in `config/profiles/raspberry`. The split profile prevents the main host and Raspberry from exposing the same environment PV names.

3 Before starting

Run normal operator commands from a terminal opened inside the Ubuntu graphical desktop or VNC session. A plain non-graphical SSH shell cannot launch the normal Phoebus workflow.

Enter the repository:

```
cd ~/SlowControl/app/bdx-slow-control
```

If an installed command is not available in the shell:

```
source .venv/bin/activate
```

The local file `config/runtime.env` must identify the main IOC host. Create it from the template if necessary:

```
cp config/runtime.env.example config/runtime.env
```

For the current deployment it must contain:

```
BDX_MAIN_HOST=172.22.50.2
```

Do not use 127.0.0.1 for normal operation. Remote Channel Access clients cannot reach a loopback-only IOC.

4 Starting the system

4.1 Raspberry environment IOC

The main startup command checks whether `BDX:ENV:TEMP:T00:VALUE` is reachable but does not start the Raspberry service automatically. Start the remote service when required:

```
start-bdx-raspberry-ioc
```

Force a restart with:

```
start-bdx-raspberry-ioc --restart
```

The installed command is declared in `pyproject.toml` and implemented by `src/bdx_slow_control/operator_commands.py`. It connects through SSH, runs `systemctl start` or `systemctl restart` for the remote `bdx-environment-ioc` service, and waits for the readiness PV. If it fails, inspect the remote service:

```
ssh -t pi@172.22.50.10 \  
  sudo journalctl -u bdx-environment-ioc -n 100 --no-pager
```

4.2 Complete operator stack

```
bdx_slow_control_start overview
```

The command:

1. resolves the repository and validates the main-host address, caproto client, Phoebus installation, and graphical terminal;
2. reports the independent Archiver Appliance endpoint health in read-only mode;
3. checks the Raspberry environment IOC and reports a warning if unavailable;
4. opens a terminal for the main aggregated IOC unless one is already listening;
5. waits for the IOC and a representative PV in the Phoebus terminal workflow;
6. launches Phoebus on the requested display.

The command never starts, stops, audits, repairs, or registers PVs in the Archiver Appliance. Use `bdx_archiver_start`, `bdx_archiver_audit`, `bdx_archiver_repair`, or `bdx_archiver_kill` explicitly when those actions are required.

The installed entry point is declared in `pyproject.toml` and calls `src/bdx_slow_control/operator_startup.py`. The Phoebus terminal workflow sources `scripts/start_bdx_stack.sh`; the main IOC command executed is `bdx-prototype-ioc`.

Startup does not enable PSU outputs, start or stop the chiller, or write setpoints. A normal launch leaves a **BDX Main IOC** terminal open when a new IOC is started and opens a separate **BDX Phoebus** terminal for the GUI workflow.

Open a different page initially with:

```
bdx_slow_control_start psu
bdx_slow_control_start chiller
bdx_slow_control_start environment
bdx_slow_control_start trends
```

When the IOC is already running and only Phoebus must be reopened:

```
./scripts/launch_phoebus.sh overview
```

5 Using Phoebus

Display	Purpose
overview.bob	Overall status and navigation
psu.bob	Routine LV1/LV2 operation with staged requests and live plots
psu_expert.bob	Complete PSU PV table and direct compatibility controls
chiller.bob	Routine chiller operation and live temperature plots
chiller_expert.bob	Complete chiller PV table and expert controls
environment.bob	Environmental sensor status and values
global.bob	Global state, update timing, interlock test, reset, and all-off
trends.bob	Principal live and archived trends
all_pvs.bob	Complete configured PV catalog

Operator pages expose the normal workflow. Expert pages expose the complete writable PV surface and should be used only when the direct behavior is understood.

Before issuing a command, verify:

- IOC_STATE is RUNNING;
- COMM_OK is true;
- COMM_STATUS is OK for hardware or SIMULATION for a simulator;
- LAST_UPDATE continues changing;
- ERROR_CODE is zero and ERROR_MESSAGE is empty.

A visible value may be stale. If communication is unhealthy, do not assume the displayed value represents the present hardware state.

6 Operating the low-voltage supplies

The PSU operator page uses staged request PVs. Editing a request field does not immediately write the instrument.

For the intended device and channel:

1. confirm the device label, channel, communication status, and output state;

2. enter the target in `VOLTAGE_REQUEST`;
3. enter the limit in `CURRENT_LIMIT_REQUEST`;
4. check both values and press the confirmed **Apply** button;
5. verify `APPLY_STATUS` and `APPLY_MESSAGE`;
6. verify `VOLTAGE_SET_RBV`, `VOLTAGE_RBV`, and `CURRENT_LIMIT_RBV`.

The IOC validates voltage and current together before writing either value. The default software envelope is 0–60 V, 0–20 A, and a maximum requested voltage-current product of 420 W. A profile may define stricter device or channel limits.

Requests and readbacks

A request or writable setpoint indicates what was entered or accepted by the IOC. A field ending in `_RBV` is updated from a device query and represents the hardware readback. Always verify the readback after a command.

Use the confirmed output control to enable or disable a channel. After the command, verify `OUTPUT_RBV` and the measured voltage/current rather than relying only on the button state.

Device and global all-off commands are available in Phoebus. They act through the configured software drivers and depend on working communication. They do not replace hardware emergency-off systems.

7 Operating the chiller

To apply a temperature setpoint:

1. verify communication and confirm that temperatures are updating;
2. enter the target in `SETPOINT_REQUEST`;
3. press the confirmed setpoint apply control;
4. verify the apply status and applied setpoint readback;
5. monitor controlled temperature, bath temperature, and deviation state.

The current configured setpoint range is 5–40 degrees C. The default deviation thresholds are 0.2 degrees C for warning and 0.5 degrees C for alarm.

The chiller does not start automatically when the IOC starts. Use the confirmed **Start** or **Stop** control and verify the running-state readback and subsequent temperature evolution.

Pressure and external-temperature measurements are disabled in the current main-server profile because they are not available from the present setup. Disabled measurements are not queried and are not shown on the operator page.

8 Environmental temperatures

The Raspberry Pi exposes four MCP9808 temperature PVs:

```
BDX:ENV:TEMP:T00:VALUE
BDX:ENV:TEMP:T01:VALUE
```

```
BDX:ENV:TEMP:T02:VALUE
BDX:ENV:TEMP:T03:VALUE
```

Read one value from the main host with:

```
caproto-get BDX:ENV:TEMP:T00:VALUE
```

The detailed Raspberry installation, I2C, diagnostic, systemd, and replacement-host procedure is maintained in `docs/raspberry.md`.

9 Viewing archived history

Phoebus Data Browser combines live Channel Access data with Archiver Appliance retrieval when the PV is registered and actively sampled. Operator pages contain principal plots and **Full history** actions for archived PVs.

History exists only from the time a PV was successfully registered and sampled. Missing earlier samples cannot be reconstructed. If live values are present but history is empty, run `bdx_archiver_audit`, then check the retrieval service, selected time range, and clock synchronization. Use `bdx_archiver_repair` only when a catalog repair is actually required.

10 Operator command implementation reference

The installed command names are declared in `pyproject.toml`. Python entry points validate arguments and dispatch to the following implementation files:

Command	Python entry point	Script or implementation used
<code>bdx_slow_control_start</code>	<code>operator_startup.py</code>	<code>sources/scripts/start_bdx_stack.sh</code> ; starts <code>bdx-prototype-ioc</code> and launches Phoebus
<code>bdx_archiver_start</code>	<code>operator_startup.py</code>	<code>sources/scripts/start_bdx_stack.sh</code> and calls its Archiver start/validation functions
<code>bdx_archiver_audit</code>	<code>operator_startup.py</code>	<code>deploy/archiver-appliance/scripts/repair-archiver.sh--audit-only</code>
<code>bdx_archiver_repair</code>	<code>operator_startup.py</code>	<code>deploy/archiver-appliance/scripts/repair-archiver.sh</code>
<code>bdx_slow_control_kill</code>	<code>operator_commands.py</code>	<code>scripts/kill_slow_control_all.sh</code>
<code>bdx_slow_control_kill_ioc</code>	<code>operator_commands.py</code>	<code>scripts/kill_slow_control_ioc.sh</code>
<code>bdx_slow_control_kill_phoebus</code>	<code>operator_commands.py</code>	<code>scripts/kill_slow_control_phoebus.sh</code>
<code>bdx_archiver_kill</code>	<code>operator_commands.py</code>	<code>scripts/kill_slow_control_archiver.sh</code>

Command	Python entry point	Script or implementation used
<code>start-bdx-raspberry-ioc</code>	<code>operator_commands.py</code>	SSH plus remote <code>systemctl start/restart</code> <code>bdx-environment-ioc</code>

11 Stopping the software

Stop individual components with:

```
bdx_slow_control_kill_phoebus
bdx_archiver_kill
bdx_slow_control_kill_ioc
```

Stop the normal local slow-control stack with:

```
bdx_slow_control_kill
```

This stops Phoebus and the main IOC but deliberately leaves the independently managed Archiver Appliance running. Normal shutdown is graceful. Use `-force` only when a recognized process does not terminate normally.

Software shutdown does not make hardware safe

Stopping Phoebus, Archiver Appliance, or the IOC does not switch off PSU outputs, send an automatic all-off, stop the chiller, or change setpoints. Put the equipment in the intended state before stopping the IOC and verify that state directly.

12 Troubleshooting reference

Symptom	Check or action
Installed command not found	Activate the environment with <code>source .venv/bin/activate</code> .
No graphical display	Run the startup command from the Ubuntu desktop or VNC session and confirm <code>DISPLAY</code> or <code>WAYLAND_DISPLAY</code> is set.
Runtime environment missing	Create <code>config/runtime.env</code> from the example and set <code>BDX_MAIN_HOST=172.22.50.2</code> .
Phoebus launcher missing	Set <code>BDX_PHOEBUS_HOME</code> or pass <code>-phoebus-home</code> ; verify that <code>phoebus.sh</code> exists.
Raspberry unavailable	Run <code>start-bdx-raspberry-ioc</code> and inspect the remote systemd journal if it fails.
LV1 or LV2 unavailable	Check cabling and front-panel network configuration, then run <code>nc -vz 172.22.50.20 9221</code> or <code>nc -vz 172.22.50.21 9221</code> .
Chiller unavailable	Run <code>nc -vz 172.22.50.60 54321</code> and inspect the main IOC terminal.
PV not found	Confirm the correct Channel Access address list, the relevant IOC is running, and no duplicate IOC exposes the same PV names.

Symptom	Check or action
Archiver absent during slow-control startup	Run <code>bdx_archiver_start</code> ; <code>bdx_slow_control_start</code> intentionally reports Archiver health read-only and continues with live control.
Live values but no history	Run <code>bdx_archiver_audit</code> ; confirm retrieval availability, selected time range, and synchronized clocks; repair only when required.
Display appears stale	Check <code>LAST_UPDATE</code> , <code>COMM_OK</code> , and the relevant IOC terminal.

Part II

Developer guide

13 Scope

This manual describes how the BDX slow-control repository is structured, configured, tested, extended, deployed, and documented. It is intended for developers and maintainers who may modify IOC behavior, add a hardware backend, introduce a subsystem, regenerate Phoebus resources, or prepare a stable release.

The repository is [vallarinosimone1992/bdx-slow-control](https://github.com/vallarinosimone1992/bdx-slow-control). Software-facing material is maintained in English. The normal development branch is `devel`; validated stable snapshots are promoted to `main`.

Source of truth

The checked-out code and JSON profiles are authoritative. This manual explains the design and maintenance workflow, but hardware addresses, limits, commands, and available PVs must be verified against the branch being deployed.

14 Design goals

The prototype is deliberately small in device count but should reproduce the final detector control model at the behavioral level. The principal design goals are:

- modular IOC groups and hardware-independent driver interfaces;
- stable, explicit EPICS PV naming;
- separation of requested values, writable setpoints, and device readbacks;
- communication health, stale-data awareness, and actionable error reporting;
- simulation backends for development and regression tests;
- safe operator workflows with explicit confirmation and staged requests;
- generated Phoebus displays synchronized with the configured PV database;
- compatibility with a single-host prototype and future split-host deployment;

- integration with Archiver Appliance, alarm infrastructure, and DAQ coordination.

Backward compatibility with the earlier monolithic `test_ioc.py` execution model is intentionally not a design constraint.

15 Architecture

15.1 Layering

```
Hardware or simulator
|
Driver interface and implementation
|
Subsystem IOC group
|
caproto Channel Access server
|
Phoebus / Archiver / alarms / DAQ coordination
```

Transport-specific communication belongs in the driver layer. IOC classes expose the EPICS contract, validate operator requests, poll drivers, and translate driver state into PV readbacks. Phoebus must not contain safety logic required for autonomous protection.

15.2 Runtime model

Each subsystem has an independent builder and can be run as an individual IOC command. The default prototype combines all JSON files present in one profile directory into one caproto server process:

```
bdx-prototype-ioc
|- global IOC group
|- PSU device and channel IOC groups
|- chiller IOC group
|- optional environment IOC groups
|- optional HV IOC groups
'- optional DAQ IOC groups
```

The aggregated process avoids ambiguous Channel Access UDP search behavior when several caproto servers share the same host and interface. Individual commands remain available for focused development and future distribution across separate hosts.

15.3 Current deployed split

The current prototype is a two-host deployment:

- the main Ubuntu host runs `bdx-prototype-ioc` with `config/profiles/default`;
- the Raspberry Pi runs only `bdx-environment-ioc` with `config/profiles/raspberry/environment.json`;
- a display-only catalog combines both profile views for Phoebus without starting a duplicate environment IOC on the main host.

Two Channel Access servers must never expose the same PV name.

16 Repository layout

```

bdx-slow-control/
|- config/
|  |- deployment/      host/network deployment configuration
|  |- display-catalogs/ display-only composition catalogs
|  |- examples/        non-installed configuration examples
|  '- profiles/        runnable IOC profiles
|- deploy/             external service deployment infrastructure
|- docs/               focused architecture and deployment notes
|- documentation/      LaTeX operator and developer manuals
|- phoebus/
|  |- displays/        generated .bob, .plt, and PV table resources
|  '- settings/        Phoebus settings templates
|- scripts/            bootstrap, launch, installation, and shutdown scripts
|- src/bdx_slow_control/
|  |- drivers/         interfaces, simulations, and hardware backends
|  '- iocs/            caproto IOC groups
|- systemd/           service examples
|- tests/             unit, integration, and display-contract tests
|- pyproject.toml      package metadata and console entry points
'- README.md           repository overview and current procedures

```

Focused documents such as `docs/raspberry.md` and `deploy/archiver-appliance/README.md` remain the detailed source for those deployments. The manuals summarize and link to them rather than duplicating every command.

17 Development environment

17.1 Requirements

The Python package requires Python 3.10 or newer. Runtime dependencies are caproto 1.x and pyserial 3.x. The development extras include pytest and Ruff. Phoebus is installed separately.

A typical checkout setup is:

```

git clone https://github.com/vallarinosimone1992/bdx-slow-control.git
cd bdx-slow-control
git switch devel
./scripts/bootstrap.sh
source .venv/bin/activate

```

The bootstrap script installs the package in editable mode and regenerates Phoebus resources from the configured PV database.

17.2 Useful checks

```

source .venv/bin/activate
pytest
ruff check src tests
bdx-pv-list --config-dir config/profiles/default

```

Run tests before and after a change. A new driver or PV should be accompanied by targeted tests, not only a manual hardware trial.

18 Configuration model

18.1 Profiles

A profile directory contains one JSON file per enabled subsystem. The aggregated prototype iterates over the registered builders and includes only files that exist:

```
config/profiles/default/  
  global.json  
  psu.json  
  chiller.json
```

The full simulated profile is in `config/profiles/prototype`. Raspberry hardware uses `config/profiles/raspberry`.

The command:

```
bdx-prototype-ioc --config-dir config/profiles/default
```

builds all supported subsystem files found in that directory. Duplicate PV names are rejected. All included subsystem configurations must resolve to the same server interface list.

18.2 Common server settings

A typical JSON file contains:

```
{  
  "server": {  
    "interfaces": ["0.0.0.0"],  
    "poll_interval": 1.0,  
    "log_pv_names": false  
  }  
}
```

`BDX_EPICS_INTERFACE` overrides `server.interfaces` at runtime. This is the preferred mechanism on a host with more than one network interface.

The configured polling interval must be positive. In the aggregated prototype, all managed IOC groups share one `RuntimeSettings` object; the global IOC can change the effective update period at runtime within configured bounds.

18.3 Device entries

Hardware-specific addresses, ports, timeouts, channel lists, optional features, and limits belong in JSON. IOC classes must not hard-code those deployment values.

Example PSU device entry:

```
{  
  "name": "LV1",  
  "prefix": "BDX:PSU:LV1:",  
  "mode": "hardware",  
  "driver": "cpx400dp",  
  "host": "172.22.50.20",  
  "port": 9221,  
  "timeout": 3.0,  
  "channels": [1, 2],  
  "software_limits": {  
    "minimum_voltage": 0.0,  
  }  
}
```

```

    "maximum_voltage": 60.0,
    "minimum_current_limit": 0.0,
    "maximum_current_limit": 20.0,
    "maximum_power": 420.0
  }
}

```

Device limits may be overridden per channel through `channel_limits`. Validate a new profile with the simulated or disconnected startup path before connecting it to hardware.

18.4 Runtime environment

Machine-specific values are not committed in `config/runtime.env`. Create it from `config/runtime.env.example` and set the operational main IOC address:

```
BDX_MAIN_HOST=172.22.50.2
```

Do not store credentials, Wi-Fi secrets, private keys, downloaded third-party runtimes, logs, databases, or archive data in the repository.

19 PV contract

19.1 Naming convention

The default hierarchy is:

```
BDX:<SUBSYSTEM>:<DEVICE>:<CHANNEL>:<FIELD>
```

Examples:

```

BDX:PSU:LV1:CH1:VOLTAGE_REQUEST
BDX:PSU:LV1:CH1:VOLTAGE_RBV
BDX:CHILLER:CHILLER1:SETPOINT_REQUEST
BDX:ENV:TEMP:T00:VALUE
BDX:DAQ:CRATE01:READY
BDX:GLOBAL:INTERLOCK_ACTIVE

```

Use these suffixes consistently:

Suffix	Meaning
<code>_RBV</code>	Read-only value obtained from device or IOC state
<code>_SET</code>	Direct writable setpoint or compatibility control
<code>_REQUEST</code>	Operator-staged value not yet applied to hardware
<code>_CMD</code>	Momentary command, normally self-resetting
<code>_STATUS</code>	Textual outcome or state
<code>_OK</code>	Boolean status suited to indicators and logic

A successful write means the IOC accepted the command path; it does not prove that the physical device reached the requested state. Update readbacks only from driver-reported state.

19.2 Managed IOC health PVs

Every class derived from `ManagedIOC` exposes:

```
HEARTBEAT
IOC_STATE
COMM_STATUS
COMM_OK
LAST_UPDATE
ERROR_CODE
ERROR_MESSAGE
SIMULATION
CLEAR_ERROR_CMD
```

At startup, `IOC_STATE` changes to `RUNNING`. The heartbeat loop performs a driver communication check, polls subsystem state, updates health PVs, and sleeps for the shared runtime period. Communication failures set `COMM_OK=false` and preserve an actionable error message.

The baseline communication states are `STARTING`, `OK`, `DISCONNECTED`, `TIMEOUT`, `INVALID_DATA`, `DEVICE_ERROR`, `DISABLED`, and `SIMULATION`. New drivers should map failures consistently rather than inventing display-specific semantics.

20 Driver layer

20.1 Interfaces and state objects

Hardware-independent interfaces are defined in `src/bdx_slow_control/drivers/base.py`. State is returned with immutable dataclasses such as `PowerChannelState`, `ChillerState`, and `DaqCrateState`.

Current abstract interfaces include:

- `PowerSupplyDriver`;
- `HighVoltageDriver`;
- `ChillerDriver`;
- `SensorDriver`;
- `DaqCrateDriver`.

All drivers implement `ping()`. Read methods return complete state objects. Write methods perform one explicit device action and raise an exception when the operation cannot be completed or verified according to the driver contract.

20.2 Simulation first

A subsystem should have a simulation backend before or together with its hardware implementation. Simulation enables:

- IOC and display development without equipment;
- deterministic validation of writable PV behavior;
- fault and interlock testing;
- continuous tests on machines without the hardware network.

Set `driver.simulation=true` for simulation implementations so `SIMULATION` and `COMM_STATUS` are correct.

20.3 Adding a hardware backend

Use the following sequence.

1. Identify the existing abstract interface and state dataclass. Extend them only when the new device requires information that is meaningful at the subsystem contract level.
2. Add the implementation under `src/bdx_slow_control/drivers/hardware/`. Keep socket, serial, protocol parsing, locking, timeouts, and device command strings inside this module.
3. Provide a small builder function that validates required JSON fields and returns the driver instance.
4. Register the driver name in `src/bdx_slow_control/drivers/factory.py`. Unsupported names must fail with an explicit configuration error.
5. Add a JSON example or operational profile entry. Do not embed host addresses or channel counts in IOC code.
6. Add unit tests for command formatting, parsing, limit behavior, timeouts, and representative failures. Use fake transports or local test servers where possible.
7. Run the subsystem IOC against simulation, then perform read-only hardware checks, then controlled write tests under an approved procedure.

Event-loop behavior

A slow blocking hardware call must not stall unrelated caproto Channel Access traffic. Use the established driver/IOC pattern for offloading and serialization when a device library or network protocol is blocking.

21 IOC layer

21.1 ManagedIOC

`src/bdx_slow_control/iocs/common.py` defines `ManagedIOC`. A subsystem IOC should normally:

- declare PVs with caproto `pvproperty`;
- implement `poll_device()` to write readbacks from one coherent driver state;
- implement putters for explicit writes;
- catch write failures, call `mark_failure()`, and re-raise or return a rejected status as appropriate;
- avoid transport-specific code and deployment constants.

21.2 Staged operator requests

Routine hardware changes should prefer request/apply semantics. The low-voltage IOC is the reference pattern:

1. request PVs are initialized from the first successful device read;
2. the operator edits voltage and current requests without immediate hardware writes;

3. `APPLY_CMD` validates both values together;
4. the driver applies the request;
5. the IOC rereads the device and updates readbacks;
6. `APPLY_STATUS` and `APPLY_MESSAGE` record the outcome.

If a multi-step hardware write can partially succeed, the failure message must state that possibility and the IOC should reread the device before reporting the final state.

21.3 Commands

Momentary command PVs should normally return to false after execution. Confirmed actions in Phoebus are required for operations such as output enable, chiller start, all-off, simulated interlock injection, and interlock reset.

Global or device all-off callbacks are registered in `PrototypeContext`. A reset clears the latched global interlock state but must not automatically restore outputs.

22 Builders and CLI entry points

`src/bdx_slow_control/builders.py` converts validated JSON objects into IOC groups and PV databases. Driver selection occurs through the factory. `src/bdx_slow_control/prototype.py` combines available subsystem files, checks for duplicate PVs, and enforces compatible server interfaces.

Console scripts are declared in `pyproject.toml`. The principal development commands are:

```
bdx-prototype-ioc --config-dir config/profiles/default
bdx-psu-ioc --config config/profiles/default/psu.json
bdx-chiller-ioc --config config/profiles/default/chiller.json
bdx-environment-ioc --config config/profiles/raspberry/environment.json
bdx-pv-list --config-dir config/profiles/default
bdx-generate-displays --config-dir config/profiles/default \
    --output-dir phoebus/displays
```

22.1 Adding a new subsystem

A new subsystem normally requires all of the following:

1. a state model and abstract driver interface in the driver layer;
2. a simulation implementation;
3. one or more IOC groups under `src/bdx_slow_control/iocs/`;
4. a builder function in `builders.py`;
5. a driver-factory function or registration;
6. a key in the BUILDERS mapping;
7. an individual CLI entry point when focused execution is useful;
8. a console-script entry in `pyproject.toml`;
9. one or more JSON profiles;

10. generated Phoebus coverage and tests;
11. Archiver PV-list updates when the subsystem must be archived;
12. documentation updates.

The subsystem filename used by the aggregated prototype must match its **BUILDERS** key, for example `psu` maps to `psu.json`.

23 Phoebus generation

23.1 Generated resources

`bdx-generate-displays` reads the actual configured caproto PV database. This avoids a manually maintained display contract that can drift from code. Generated resources include operator displays, expert displays, Data Browser plot files, `all_pvs.bob`, and a valid Phoebus PV Table file.

Operator pages are intentionally selective. Expert pages provide complete PV tables and direct compatibility controls. Every writable PV must have an appropriate generated control, and every configured PV must appear in the full PV view.

23.2 Common generation commands

Default operational profile:

```
bdx-generate-displays \  
  --config-dir config/profiles/default \  
  --output-dir phoebus/displays
```

Full simulated prototype:

```
bdx-generate-displays \  
  --config-dir config/profiles/prototype \  
  --output-dir phoebus/displays
```

Deployed two-host display catalog:

```
bdx-generate-displays \  
  --catalog config/display-catalogs/deployed-prototype.json \  
  --output-dir phoebus/displays
```

Update only one subsystem when appropriate:

```
bdx-generate-displays \  
  --config-dir config/profiles/default \  
  --output-dir phoebus/displays \  
  --only psu
```

Review generated diffs before committing. Unrelated displays should not change during a subsystem-only modification.

23.3 Plot and archive settings

The generator recognizes these principal environment variables:

- `BDX_TREND_RANGE`, `BDX_TREND_SCAN_PERIOD`, and `BDX_TREND_RING_SIZE`;
- `BDX_TREND_ARCHIVE_REQUEST`;

- `BDX_ARCHIVER_ENABLED`, `BDX_ARCHIVER_URL`, and `BDX_ARCHIVER_NAME`.

Long historical ranges may use optimized Archiver retrieval. Live-only resources can be generated with `BDX_ARCHIVER_ENABLED=false`. The archive URL embedded in a display must correspond to the deployment being documented.

24 Archiver Appliance integration

Repository-owned Archiver infrastructure is under `deploy/archiver-appliance`. It contains scripts, templates, checksums, policies, PV lists, service examples, and tests, but not third-party release binaries, credentials, logs, databases, or archived samples.

The operational catalog intentionally contains 18 essential measurements and applied setpoints from the PSU, chiller, and Raspberry environment IOC. Heartbeat and `LAST_UPDATE` diagnostics are excluded because every archived sample already carries its EPICS event timestamp.

The Archiver lifecycle is independent of the main IOC and Phoebus lifecycle. Use:

```
bdx_archiver_start
bdx_archiver_audit
bdx_archiver_repair
bdx_archiver_kill
```

`bdx_archiver_start` is declared in `pyproject.toml` and implemented by `src/bdx_slow_control/operator_startup.py`. It sources `scripts/start_bdx_stack.sh`, starts and validates the four Archiver Appliance components, and, unless `-no-repair` is supplied, audits and selectively repairs the required catalog. It never starts an IOC and never launches Phoebus.

`bdx_archiver_audit` and `bdx_archiver_repair` are also implemented by `src/bdx_slow_control/operator_startup.py`. Both dispatch to `deploy/archiver-appliance/scripts/repair-archiver.sh`; audit adds `-audit-only`, while repair performs the controlled catalog changes. Existing healthy entries are preserved and out-of-scope registrations are not removed.

`bdx_archiver_kill` is implemented by `src/bdx_slow_control/operator_commands.py`, which dispatches to `scripts/kill_slow_control_archiver.sh`.

When adding or renaming a PV that should be archived:

1. update the appropriate repository PV list and policy;
2. run offline tests and PV-list validation;
3. run `bdx_archiver_audit` to inspect the deployed state;
4. run `bdx_archiver_repair` when a controlled repair is required;
5. confirm `connectionState=true` and the expected effective policy;
6. verify live and historical retrieval in Phoebus;
7. document that samples exist only from the successful registration time onward.

Keep host clocks synchronized. Data Browser retrieval is timestamp-based, and clock skew can make recent samples appear absent. Archiver faults do not disable live Channel Access control: Phoebus remains usable for live operation while archive recovery is performed.

25 Operator command layer

The canonical installed Ubuntu commands are declared in `pyproject.toml`. They provide a constrained operator workflow while retaining explicit separation between slow-control, Archiver, and Raspberry lifecycle actions.

Command	Python entry point	Script or implementation used
<code>bdx_slow_control_start</code>	<code>src/bdx_slow_control/operator_startup.py</code>	sources <code>scripts/start_bdx_stack.sh</code> ; starts or reuses <code>bdx-prototype-ioc</code> , reports Archiver health read-only, and launches Phoebus
<code>bdx_slow_control_kill</code>	<code>src/bdx_slow_control/operator_commands.py</code>	<code>scripts/kill_slow_control_all.sh</code>
<code>bdx_slow_control_kill_ioc</code>	<code>src/bdx_slow_control/operator_commands.py</code>	<code>scripts/kill_slow_control_ioc.sh</code>
<code>bdx_slow_control_kill_phoebus</code>	<code>src/bdx_slow_control/operator_commands.py</code>	<code>scripts/kill_slow_control_phoebus.sh</code>
<code>bdx_archiver_start</code>	<code>src/bdx_slow_control/operator_startup.py</code>	Archiver functions sourced from <code>scripts/start_bdx_stack.sh</code>
<code>bdx_archiver_repair</code>	<code>src/bdx_slow_control/operator_startup.py</code>	<code>deploy/archiver-appliance/scripts/repair-archiver.sh</code>
<code>bdx_archiver_audit</code>	<code>src/bdx_slow_control/operator_startup.py</code>	<code>deploy/archiver-appliance/scripts/repair-archiver.sh--audit-only</code>
<code>bdx_archiver_kill</code>	<code>src/bdx_slow_control/operator_commands.py</code>	<code>scripts/kill_slow_control_archiver.sh</code>
<code>start-bdx-raspberry-ioc</code>	<code>src/bdx_slow_control/operator_commands.py</code>	SSH plus remote <code>systemctl start/restart bdx-environment-ioc</code>

The normal `bdx_slow_control_start` path resolves the repository root, validates `BDX_MAIN_HOST`, verifies the caproto client, checks the Raspberry readiness PV, opens a dedicated IOC terminal if required, reports Archiver endpoint health without mutating the service, waits for IOC readiness in the Phoebus terminal workflow, and launches Phoebus. It must be run from a graphical Ubuntu session.

In particular, `bdx_slow_control_start` never starts, stops, audits, repairs, or registers PVs in the Archiver Appliance. This invariant prevents routine GUI startup from changing the independent archive service.

`bdx_slow_control_kill` stops only the project Phoebus session and main IOC. It deliberately leaves the independently managed Archiver Appliance running. Shutdown commands target only recognized BDX slow-control processes, use graceful termination by default, and reserve forced killing for an explicit `-force`. Software process shutdown must remain separate from hardware state changes.

The legacy names `start_slow_control`, `kill_slow_control`, `start_archiver`, `kill_archiver`,

and the older `bdx_slow_control_*_archiver` commands remain temporary compatibility aliases. New deployments, automation, and documentation must use the canonical names above.

When changing operator commands, update both the `pyproject.toml` entry-point mapping and this command-reference table. Add regression tests for process discovery, reused PID files, exact command matching, read-only Archiver behavior during slow-control startup, and protection of unrelated BDX DAQ or Phoebus sessions.

26 Raspberry environment IOC

The Raspberry Pi runs only the MCP9808 environment IOC. The canonical configuration files are:

- `config/profiles/raspberry/environment.json`;
- `config/profiles/raspberry/bdx.env`;
- `config/deployment/raspberry-network.env`.

The current hardware uses I2C bus `/dev/i2c-6`, GPIO22/GPIO23, addresses `0x18` through `0x1B`, and `172.22.50.10/24` on Ethernet. Network configuration and software installation are intentionally separate operations.

Before enabling the service, run the repository diagnostic against the installed JSON profile. The detailed installation, I2C overlay, diagnostic, systemd, and replacement-host procedure is maintained in `docs/raspberry.md`.

27 Run temperature export

The command `bdx_run_temperature_export` retrieves the configured Raspberry temperature PVs from EPICS Archiver Appliance over the interval defined by a DAQ run information file. It can produce a JSON summary and, when the optional export dependencies are installed, a ROOT file for downstream DAQ analysis.

ROOT support is intentionally optional so IOC-only hosts and the Raspberry Pi do not acquire NumPy or uproot. Install it on the DAQ or analysis host with:

```
python -m pip install -e '[export]'
```

The default configuration is `config/run_temperature_export.json`. It defines the DAQ run root, run-directory and information-file patterns, the Europe/Rome run time zone, the Archiver retrieval endpoint and timeout, the output path, histogram bin width, missing-alarm sentinel, and the temperature PV list.

Create the final ROOT file with:

```
bdx_run_temperature_export 260714 --write-root
```

The output is written atomically to `<run_directory>/OTHER/SlowControl_<run_id>.root`. The `OTHER` directory must already exist. An existing output is replaced only when `-overwrite` is supplied.

The ROOT output contains:

- a `temperature_samples` TTree with normalized timestamps, values, status, and severity;
- mean and sample-count TH1D histograms for each configured sensor;

- a `metadata` `TObjString` containing UTF-8 JSON describing the run interval, configuration, warnings, and output provenance.

Missing Archiver status or severity fields use the configured `missing_alarm_value` sentinel, currently `-1`; the sentinel is also recorded in the metadata. Retrieval is timestamp-based, therefore the DAQ host, IOC hosts, and Archiver host must use synchronized clocks and the run information file must be interpreted in the configured time zone.

When modifying this workflow, preserve atomic output semantics, explicit overwrite protection, strict Archiver-response validation, and tests covering run-file parsing, time-zone conversion, empty intervals, malformed samples, ROOT structure, and metadata consistency.

28 Testing strategy

28.1 Required test categories

The existing suite checks driver behavior, PV naming and aggregation, update-period limits, XML validity of generated `.bob` resources, complete PV coverage, and controls for writable PVs. Extend the suite when changing any of those contracts.

For a new feature, include as applicable:

- pure parsing and command-format tests;
- driver tests with fake transports;
- IOC putter and polling tests;
- configuration validation tests;
- duplicate-PV and profile-composition tests;
- generated-display contract tests;
- operator script tests with fake processes or commands;
- a documented manual hardware acceptance test.

28.2 Pre-commit checklist

```
source .venv/bin/activate
ruff check src tests
pytest
bdx-pv-list --config-dir config/profiles/default > /tmp/bdx-default.pvs
bdx-generate-displays --config-dir config/profiles/default \
  --output-dir phoebus/displays

git status --short
git diff --check
```

Inspect generated XML diffs rather than committing them blindly. Confirm that no local runtime file, credentials, logs, databases, or archive data are staged.

29 Logging and diagnostics

The package configures Python logging centrally. Set `BDX_LOG_LEVEL` to increase verbosity during diagnosis:

```
BDX_LOG_LEVEL=DEBUG bdx-prototype-ioc \  
--config-dir config/profiles/default
```

Useful diagnostic principles:

- start with read-only network checks;
- distinguish a failed TCP connection from a protocol parse failure;
- report the device, channel, attempted operation, and underlying exception;
- avoid repeated identical log floods during a persistent poll failure;
- log recovery when communication becomes healthy again;
- never print credentials or private keys.

For Channel Access checks, set an explicit address list and disable automatic broadcast discovery when diagnosing a split-host system:

```
export EPICS_CA_ADDR_LIST="172.22.50.2 172.22.50.10"  
export EPICS_CA_AUTO_ADDR_LIST=NO  
caproto-get BDX:GLOBAL:SYSTEM_STATE  
caproto-get BDX:ENV:TEMP:T00:VALUE
```

30 Git workflow and stable releases

Routine development should occur on `devel`. Use focused commits with tests and documentation in the same change. Promote to `main` only after the intended hardware/simulation workflow and generated resources have been validated.

Recommended workflow:

```
git switch devel  
git pull --ff-only  
  
# edit, test, regenerate, document  
  
git status --short  
git diff --check  
git add <reviewed paths>  
git commit -m "Describe the focused change"  
git push origin devel
```

For a stable promotion:

1. ensure `devel` is clean and pushed;
2. run the complete automated test suite;
3. perform the relevant hardware acceptance checks;
4. verify generated Phoebus resources and Archiver lists;
5. update manuals and version/revision notes;
6. build and inspect the two PDF manuals;

7. merge or fast-forward the validated state to `main` according to the chosen repository policy;
8. tag the release if a durable deployment reference is required.

Avoid committing directly to `main` for ordinary development.

31 Building these manuals

The LaTeX sources are in `documentation`. From the repository root:

```
make -C documentation
```

The build places the final operator and developer PDFs in the repository root:

```
BDX_Slow_Control_Quickstart.pdf
BDX_Slow_Control_Developer_Manual.pdf
```

Intermediate LaTeX files remain under `documentation/build` and are ignored or removed with:

```
make -C documentation clean
```

Before committing exported PDFs, render and inspect every page. Check for clipped code, overfull lines, broken links, stale addresses, and inconsistency with the current branch.

32 Change checklist

Use this compact checklist for modifications that affect operation or the PV contract.

1. Update or add the driver interface, simulation, hardware backend, IOC, builder, factory, CLI, and profile as required.
2. Preserve request/setpoint/readback semantics and common health PV behavior.
3. Add or update automated tests.
4. Regenerate only the affected Phoebus resources, then inspect all diffs.
5. Update Archiver PV lists for newly archived PVs.
6. Update the quickstart guide if operator behavior changes.
7. Update this manual and focused deployment documents if architecture or maintenance changes.
8. Run Ruff, pytest, PV-list generation, and display generation.
9. Perform a controlled hardware acceptance test when hardware behavior changed.
10. Commit to `devel`; promote to `main` only after validation.

A Principal environment variables

Variable	Purpose
<code>BDX_SLOW_CONTROL_ROOT</code>	Explicit repository checkout path for operator commands
<code>BDX_MAIN_HOST</code>	Operational main IOC address

Variable	Purpose
BDX_RUNTIME_ENV	Alternative runtime environment file
BDX_EPICS_INTERFACE	Channel Access server interface override
BDX_CA_ADDR_LIST	Phoebus/client Channel Access address list
BDX_CA_AUTO_ADDR_LIST	Enable or disable automatic CA address discovery
BDX_PHOEBUS_HOME	Phoebus installation directory containing <code>phoebus.sh</code>
BDX_PHOEBUS_CMD	Explicit Phoebus launcher command
BDX_PHOEBUS_DISPLAY	Initial display resource
BDX_TERMINAL	Explicit graphical terminal executable
BDX_LOG_LEVEL	Python/IOC log verbosity
BDX_STACK_RUNTIME_DIR	Runtime PID and stack state directory
BDX_RASPBERRY_SSH_HOST	SSH destination for Raspberry service control
BDX_ARCHIVER_ENABLED	Include or disable archive retrieval in generated resources
BDX_ARCHIVER_URL	Archiver retrieval endpoint
BDX_ARCHIVER_NAME	Phoebus archive source name
BDX_ARCHIVER_STRICT_CHECK	Make archive preflight failure fatal when enabled
BDX_TREND_RANGE	Default generated plot time range
BDX_TREND_SCAN_PERIOD	Live plot scan period
BDX_TREND_RING_SIZE	Live Data Browser buffer size
BDX_TREND_ARCHIVE_REQUEST	Archive retrieval optimization mode

B Principal console scripts

Command	Role
<code>bdx-prototype-ioc</code>	Run all configured local subsystem groups in one caproto server
<code>bdx-psu-ioc</code>	Run only the PSU builder/profile
<code>bdx-chiller-ioc</code>	Run only the chiller builder/profile
<code>bdx-environment-ioc</code>	Run only the environment builder/profile
<code>bdx-hv-ioc</code>	Run only the HV builder/profile
<code>bdx-daq-ioc</code>	Run only the DAQ builder/profile
<code>bdx-global-ioc</code>	Run only the global builder/profile
<code>bdx-pv-list</code>	Print or write the configured PV names
<code>bdx-generate-displays</code>	Generate Phoebus displays, plots, and PV tables
<code>bdx-environment-check</code>	Validate the configured environment sensor hardware
<code>bdx_slow_control_start</code>	Start/reuse the Ubuntu operator stack and launch Phoebus
<code>bdx_slow_control_kill</code>	Stop the project Phoebus session and main IOC
<code>bdx_slow_control_kill_ioc</code>	Stop only project-owned local IOC processes
<code>bdx_slow_control_kill_phoebus</code>	Stop only the recorded BDX Phoebus process
<code>bdx_archiver_start</code>	Start and validate the independent Archiver Appliance
<code>bdx_archiver_repair</code>	Audit and selectively repair the required Archiver catalog
<code>bdx_archiver_audit</code>	Audit the required Archiver catalog without repair
<code>bdx_archiver_kill</code>	Stop only the independent Archiver Appliance
<code>start-bdx-raspberry-ioc</code>	Control and verify the remote Raspberry environment service

